

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày cơ sở phát hiện đối tượng bằng YOLO, dữ liệu gán nhãn và phương pháp huấn luyện tinh chỉnh mô hình từ hai lớp lên ba lớp. Tiếp đó, chương mô tả cách xử lý ảnh, video, liên kết vùng đầu với xe máy, theo dõi đối tượng và xác nhận trạng thái không đội mũ. Các công nghệ triển khai và tiêu chí đánh giá được xác định để làm cơ sở thiết kế, thực nghiệm hệ thống.

2.1. Cơ sở bài toán và huấn luyện tinh chỉnh mô hình YOLO

Hệ thống kết hợp mô hình YOLO với các bước xử lý sau nhận diện. Mô hình xác định vị trí xe máy, vùng đầu và trạng thái đội mũ; quy tắc liên kết và theo dõi giúp xác định trường hợp cần ghi nhận. Phần này trình bày cơ sở phát hiện đối tượng, dữ liệu gán nhãn và cách huấn luyện tinh chỉnh từ checkpoint hai lớp sang mô hình ba lớp.

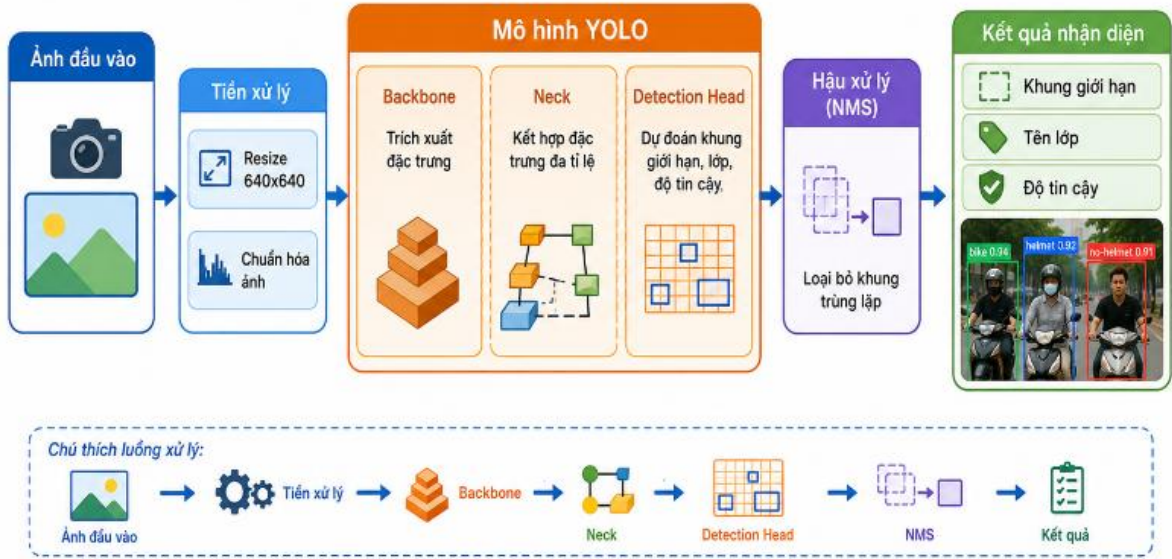
2.1.1. Giới thiệu bài toán phát hiện đối tượng và ứng dụng vào đề tài

Phát hiện đối tượng thực hiện đồng thời việc phân loại và định vị đối tượng trong ảnh. Mỗi kết quả gồm nhãn dự đoán, độ tin cậy và hộp giới hạn với tọa độ, kích thước tương ứng. Đề tài sử dụng ba lớp: *helmet* (vùng đầu có mũ), *no-helmet* (vùng đầu không mũ) và *bike* (xe máy).

Một vùng *no-helmet* chỉ cho biết trạng thái vùng đầu, chưa chứng minh người đó đang đi xe máy; vùng này có thể thuộc người đi bộ. Vì vậy, trước khi ghi nhận cần kiểm tra quan hệ với xe máy. Trên video, đối tượng còn phải được theo dõi giữa các khung hình để hạn chế tính lặp.

2.1.2. Tổng quan mô hình YOLO và các thông tin đầu ra

YOLO (*You Only Look Once*) là họ mô hình phát hiện đối tượng dựa trên học sâu, xử lý toàn bộ ảnh trong một lần suy luận và có thể dự đoán nhiều đối tượng. Kiến trúc tổng quát gồm *backbone* trích xuất đặc trưng; *neck* kết hợp đặc trưng ở nhiều tỉ lệ để hỗ trợ phát hiện đối tượng lớn, nhỏ; và *detection head* dự đoán hộp giới hạn, nhãn, độ tin cậy. Sau dự đoán, thuật toán NMS loại bớt các hộp trùng lặp, giữ kết quả phù hợp. Hình 2.1 minh họa luồng xử lý này.



Hình 2.1. Kiến trúc xử lý tổng quát của mô hình YOLO

2.1.3. Dữ liệu gán nhãn và phân chia tập huấn luyện, xác thực, kiểm thử

2.1.3.1. Dữ liệu gán nhãn theo định dạng YOLO

Mỗi ảnh huấn luyện cần tệp nhãn mô tả lớp và vị trí các đối tượng. Trong định dạng YOLO, một đối tượng chiếm một dòng theo cấu trúc:

class_id x_center y_center width height

ID lớp bắt đầu từ 0. Bốn giá trị tọa độ, kích thước được chuẩn hóa theo chiều rộng W và chiều cao H của ảnh. Nếu hộp giới hạn có các cạnh $x_{min}, y_{min}, x_{max}, y_{max}$ thì:

$$x_c = \frac{x_{min} + x_{max}}{2W}, \quad y_c = \frac{y_{min} + y_{max}}{2H}, \quad (2.1)$$

$$w = \frac{x_{max} - x_{min}}{W}, \quad h = \frac{y_{max} - y_{min}}{H}. \quad (2.2)$$

Trong đó, x_c, y_c là tọa độ tâm chuẩn hóa; w, h là chiều rộng, chiều cao chuẩn hóa. Chẳng hạn, nếu *bike* có ID 0, dòng 0 0.50 0.60 0.30 0.40 biểu diễn xe máy có tâm tại 50% chiều rộng, 60% chiều cao ảnh; hộp chiếm 30% chiều rộng và 40% chiều cao.

2.1.3.2. Tính nhất quán của nhãn

Tên lớp và ID trong tệp cấu hình phải khớp với tệp nhãn; một ID không mặc nhiên giữ cùng ý nghĩa giữa các bộ dữ liệu. Quy tắc gán nhãn cần xác định

rõ hộp đánh dấu vùng đầu, mũ hay toàn bộ người và có áp dụng cho người đi bộ hay không. Dữ liệu huấn luyện, xác thực và kiểm thử phải theo cùng quy tắc. Ngoài đúng định dạng, cần kiểm tra vị trí hộp, nhãn bị thiếu và sự tương ứng giữa ảnh với tệp nhãn; chỉ đếm số tệp không đủ bảo đảm chất lượng dữ liệu.

2.1.3.3. Phân chia dữ liệu

Dữ liệu gồm *train* để cập nhật tham số, *validation* để theo dõi chất lượng và chọn cấu hình, *checkpoint*, cùng *test* để đánh giá sau khi hoàn tất lựa chọn. Tập *test* cần độc lập; điều chỉnh mô hình liên tục theo kết quả *test* sẽ làm mất vai trò đối chứng của tập này. Với dữ liệu lấy từ video, nên tách theo video hoặc phiên ghi hình để tránh khung hình gần giống nhau xuất hiện ở nhiều tập. Notebook của đề tài chỉ đọc các tập đã khai báo trong cấu hình, không tự chia dữ liệu, nên tính độc lập phải được kiểm tra ở bước chuẩn bị.

2.1.4. Cơ sở học chuyển giao và huấn luyện tinh chỉnh

Học chuyển giao tận dụng tri thức của mô hình đã huấn luyện làm điểm khởi tạo cho nhiệm vụ mới. Huấn luyện tinh chỉnh (*fine-tuning*) tiếp tục cập nhật các tham số có thể huấn luyện, khác với việc giữ cố định mô hình như bộ trích xuất đặc trưng. Đề tài khởi tạo từ *checkpoint* nhận diện hai lớp, kế thừa các trọng số tương thích rồi huấn luyện trên dữ liệu ba lớp để bổ sung xe máy.

Mục tiêu tổng quát của quá trình huấn luyện là:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{det}(f_{\theta}(x_i), Y_i) \quad (2.3)$$

Ở đây, θ là tập tham số điều chỉnh; N là số ảnh huấn luyện; x_i là ảnh thứ i ; Y_i là tập hộp và nhãn đúng; $f_{\theta}(x_i)$ là dự đoán; $\mathcal{L}_{det}$ là hàm mất mát phát hiện đối tượng. Hàm mất mát phản ánh sai lệch về vị trí hộp và lớp dự đoán, với thành phần cụ thể tùy mô hình, phiên bản thư viện. Đề tài dùng cơ chế huấn luyện của Ultralytics, không xây dựng hàm mất mát riêng. Thuật toán cập nhật tham số qua từng nhóm dữ liệu nhằm giảm sai lệch, nhưng không bảo đảm nghiệm tối ưu toàn cục.

2.1.5. Mở rộng mô hình từ hai lớp sang ba lớp

Mô hình ban đầu nhận diện *helmet* và *no-helmet*. Phương án phát triển thêm *bike* để có vị trí phương tiện làm căn cứ liên kết với vùng đầu. *Checkpoint* hai lớp được dùng làm điểm khởi tạo cho lượt huấn luyện trên bộ dữ liệu ba lớp. Khi

số lớp thay đổi, phần dự đoán phải thích ứng với đầu ra mới; các trọng số tương thích được kế thừa, còn tham số có kích thước không phù hợp được xử lý khi khởi tạo. Cách chuyển trọng số phụ thuộc mô hình và phiên bản thư viện, cần đối chiếu nhật ký huấn luyện.

Notebook không thiết lập đóng băng toàn bộ phần trích xuất đặc trưng; các tham số có thể huấn luyện tiếp tục được điều chỉnh để học lớp xe máy và duy trì khả năng phân biệt hai trạng thái đội mũ. Cần kiểm tra lại toàn bộ ID theo cấu hình mới: nếu ID 0 từng là *helmet* nhưng nay là *bike*, nhãn cũ không thể giữ nguyên ý nghĩa.

Bổ sung *bike* chưa tự giải quyết việc phân biệt người đi bộ. Một vùng *no-helmet* chỉ trở thành ứng viên ghi nhận khi quy tắc liên kết cho thấy quan hệ phù hợp với xe máy. Sau huấn luyện, cần đánh giá từng lớp và kiểm thử cảnh có người đi bộ, nhất là khi họ đứng gần xe, để xác định mức giảm báo nhằm cùng nguy cơ bỏ sót.

2.1.6. Các tham số huấn luyện và lựa chọn checkpoint kết quả

Cấu hình huấn luyện chi phối cách cập nhật tham số, tài nguyên tính toán và khả năng thích nghi của mô hình. Bảng 2.1 tóm tắt các tham số sử dụng.

Bảng 2.1. Một số tham số huấn luyện cơ bản

Tham số	Ý nghĩa
epochs	Số lượt tối đa duyệt qua tập huấn luyện
batch	Số ảnh trong một nhóm xử lý
imgsz	Kích thước ảnh mục tiêu đưa vào mô hình
lr0	Tốc độ học ban đầu được cấu hình
patience	Số epoch chờ cải thiện trước khi dừng sớm
seed	Giá trị khởi tạo cho các quá trình ngẫu nhiên
device	Thiết bị tính toán

Trong huấn luyện, cần theo dõi cả sai số train và chất lượng validation. Nếu train tiếp tục cải thiện nhưng validation đứng yên hoặc giảm, mô hình có thể thích nghi quá mức; cơ chế dừng sớm kết thúc khi tiêu chí theo dõi không cải thiện qua số epoch chờ đã cấu hình. Checkpoint *last.pt* lưu trạng thái cuối, còn *best.pt* là trạng thái tốt nhất theo tiêu chí bộ huấn luyện theo dõi, thường dựa vào validation; do đó không nhất thiết ở epoch cuối hay có sai số train thấp nhất.

Sau khi chọn checkpoint, mô hình được đánh giá trên test. Notebook ưu tiên test và chuyển sang validation nếu cấu hình thiếu test; báo cáo phải ghi đúng tập thực dùng. Precision, Recall, F1-score và mAP đánh giá chất lượng nhận diện, còn mức giảm nhầm người đi bộ và đếm trùng phải được kiểm tra sau khi tích hợp toàn bộ quy trình.

2.2. Cơ sở xử lý ảnh, video và xác định trường hợp không đội mũ

Mô hình ba lớp cung cấp vị trí vùng đầu có mũ, vùng đầu không mũ và xe máy. Để ghi nhận trường hợp cần tìm, hệ thống còn phải chọn khung hình xử lý, liên kết vùng đầu với xe và, đối với video, theo dõi đối tượng để xác nhận trạng thái mà không đếm lặp.

2.2.1. Quy trình nhận diện trên ảnh và lấy mẫu khung hình video

2.2.1.1. Xử lý ảnh

Hệ thống kiểm tra ảnh đầu vào, chạy mô hình và nhận các hộp giới hạn, nhãn lớp, độ tin cậy. Các phát hiện được chuyển sang bước liên kết đối tượng trước khi đánh dấu, tổng hợp và lưu ảnh minh chứng. Tọa độ dùng để liên kết, vẽ và cắt minh chứng phải thuộc cùng hệ tọa độ ảnh. Vì ảnh chỉ phản ánh một thời điểm, việc xác định quan hệ chủ yếu dựa vào vị trí; người đi bộ đứng gần xe hoặc đối tượng bị che khuất vẫn có thể gây nhầm lẫn.

2.2.1.2. Xử lý video

OpenCV được dùng để đọc video bằng VideoCapture, xử lý các khung hình được chọn và ghi video kết quả bằng VideoWriter theo kích thước, tốc độ khung hình đã cấu hình. Khi cần giảm kích thước khung hình, hệ thống giữ nguyên tỷ lệ hai chiều để tránh biến dạng; đổi lại, vùng đầu nhỏ hơn có thể khó phân loại trạng thái đội mũ.

Gọi s là bước lấy mẫu. Với chỉ số khung hình bắt đầu từ 0, mô hình xử lý các khung $0, s, 2s, \dots$ $s=1$ nghĩa là xử lý toàn bộ; tăng s làm giảm số lần chạy mô hình. Nếu tốc độ khung hình ổn định là f khung hình/giây, khoảng cách thời gian giữa hai lần nhận diện và thời điểm khung thứ n là:

$$\Delta t = \frac{s}{f}, \quad t_n = \frac{n}{f} \quad (2.4)$$

Ví dụ, $f=30$ và $s=3$ cho khoảng cách 0,1 giây **trong video** giữa hai lần nhận diện, không phải thời gian máy tính xử lý. Bước lấy mẫu lớn có thể bỏ qua đối tượng xuất hiện ngắn và khiến tracking khó duy trì. Công thức thời điểm cũng cần được dùng thận trọng nếu tốc độ khung hình biến đổi hoặc thông tin

video không chính xác; OpenCV cung cấp thuộc tính tốc độ, số khung và vị trí thời gian để hỗ trợ kiểm tra. Trong triển khai hiện tại, chỉ các khung được chọn mới chạy nhận diện và cập nhật tracking, còn các khung khác vẫn được ghi vào video đầu ra.

2.2.2. Liên kết vùng đầu với xe máy theo quan hệ không gian

2.2.2.1. Cơ sở liên kết đối tượng

Đầu ra ba lớp chưa cho biết vùng đầu thuộc xe nào. Trong nhánh có xe được phát hiện, hệ thống kiểm tra tâm vùng đầu so với khu vực phía trên hộp xe, dựa trên giả định đầu người ngồi trên xe thường xuất hiện tại khu vực này. Chỉ dùng độ chồng lấp hộp giới hạn là chưa đủ, vì vùng đầu có thể nằm phía trên và không giao hộp xe tùy góc nhìn, cách gán nhãn. Vì vậy, hệ thống mở rộng hộp xe thành vùng kiểm tra.

2.2.2.2. Xác định vùng liên kết

Với xe thứ j , gọi hộp giới hạn là $B_j=(x_{1j},y_{1j},x_{2j},y_{2j})$, chiều rộng $w_j=\max(1,x_{2j}-x_{1j})$ và chiều cao $h_j=\max(1,y_{2j}-y_{1j})$. Vùng kiểm tra được xác định:

$$R_j = [x_{1j} - \alpha w_j, x_{2j} + \alpha w_j] \times [y_{1j} - \beta h_j, y_{1j} + \gamma h_j]. \quad (2.5)$$

Ở đây, α mở rộng ngang, β mở rộng lên trên, còn γ giới hạn phần phía dưới tính từ cạnh trên hộp xe. Tâm vùng đầu nằm trong R_j thì vùng đầu trở thành ứng viên liên kết. Mã hiện tại dùng $\alpha=0,2$, $\beta=0,8$, $\gamma=0,7$; đây là các hệ số kinh nghiệm cần kiểm chứng trên dữ liệu thực tế.

2.2.2.3. Lựa chọn xe phù hợp

Nếu tâm vùng đầu $C_i=(u_i,v_i)$ nằm trong vùng kiểm tra của nhiều xe, hệ thống chọn xe có khoảng cách chuẩn hóa nhỏ nhất. Điểm tham chiếu là trung điểm cạnh trên hộp xe:

$$d_{ij} = \frac{\sqrt{(u_i - \frac{(x_{1j} + x_{2j})}{2})^2 + (v_i - y_{1j})^2}}{\min(w_j, h_j)}. \quad (2.6)$$

Chuẩn hóa theo kích thước giúp so sánh khoảng cách tương đối giữa các xe. Mỗi vùng đầu được ghép với tối đa một xe, nhưng một xe có thể nhận nhiều vùng đầu, chẳng hạn khi chở nhiều người hoặc có người đứng cạnh.

2.2.2.4. Xác định trạng thái và giới hạn

Khi có các vùng đầu liên kết, mã hiện tại ưu tiên trạng thái NO_HELMET nếu ít nhất một vùng không mũ được ghép với xe; xe chưa có thông tin vùng đầu phù hợp giữ trạng thái UNKNOWN. Quy tắc này cho biết có vùng đầu không mũ liên quan đến xe, chưa phân biệt được người lái với người ngồi sau. Người đi bộ đứng sát xe cũng có thể thỏa điều kiện hình học. Do đó, liên kết không gian hỗ trợ xác định ngữ cảnh nhưng không bảo đảm kết luận về vai trò của người được phát hiện.

2.2.3. Theo dõi đối tượng bằng phương pháp Centroid Tracking

2.2.3.1. Mục đích của theo dõi đối tượng

Tracking liên kết các lần phát hiện qua khung hình và gán mã cho đối tượng để cập nhật vị trí, trạng thái, hạn chế đếm trùng. Với hộp (x_1, y_1, x_2, y_2) , tâm là:

$$C = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right).$$

Nhánh liên kết theo xe dùng tâm hộp xe; nhánh xử lý vùng đầu độc lập dùng tâm vùng đầu.

2.2.3.2. Ghép đối tượng giữa các lần cập nhật

Gọi $P_i = (p_{xi}, p_{yi})$ là tâm đối tượng đang theo dõi và $Q_j = (q_{xj}, q_{yj})$ là tâm phát hiện mới. Khoảng cách Euclid là:

$$D_{ij} = \sqrt{(p_{xi} - q_{xj})^2 + (p_{yi} - q_{yj})^2}. \quad (2.7)$$

Khi hai đối tượng có cùng trạng thái đã xác định, chi phí ghép trong mã là $0,85D_{ij}$; trường hợp khác dùng D_{ij} . Cặp chỉ hợp lệ nếu chi phí không vượt $\rho \max(W, H)$, với W, H là kích thước khung hình và ρ là hệ số khoảng cách. Các cặp hợp lệ được xếp theo chi phí tăng dần rồi chọn lần lượt, mỗi mã và phát hiện chỉ tham gia một cặp. Cách ghép tham lam này đơn giản nhưng không bảo đảm tối ưu toàn cục.

2.2.3.3. Quản lý đối tượng xuất hiện và mất dấu

Đối tượng không ghép được sẽ được theo dõi thêm trong một số lần cập nhật; phát hiện mới không ghép được nhận mã mới. Các xe đi sát, giao cắt hoặc bị che khuất có thể làm thuật toán ghép sai. Xe mất dấu lâu rồi xuất hiện lại có thể nhận mã khác và bị đếm lặp. Mã theo dõi chỉ có ý nghĩa trong một video,

không phải danh tính người hoặc phương tiện và không dùng để nhận biết cùng xe giữa các video.

2.2.4. Xác nhận trạng thái qua nhiều khung hình và hạn chế đếm trùng

2.2.4.1. Xác nhận bằng chuỗi phát hiện liên tiếp

Một khung hình nhòe, che khuất hoặc dự đoán sai có thể tạo nhân không đội mũ nhất thời. Vì vậy, với đối tượng i tại lượt cập nhật k , hệ thống duy trì số lần liên tiếp $r_i(k)$:

$$r_i^{(k)} = \begin{cases} r_i^{(k-1)} + 1, & \text{nếu đối tượng được ghép và} \\ & \text{có trạng thái NO_HELMET,} \\ 0, & \text{nếu mất ghép hoặc} \\ & \text{có trạng thái khác.} \end{cases} \quad (2.8)$$

Đối tượng mới không đội mũ bắt đầu ở 1; khi $r_i(k) \geq K$, đối tượng được xác nhận. “Liên tiếp” nghĩa là liên tiếp giữa **các khung được chạy AI**. Khung bị bỏ qua do lấy mẫu không tăng hay đặt lại bộ đếm, vì tracker không cập nhật ở khung đó.

2.2.4.2. Ví dụ minh họa

Với $K=3$, quá trình xác nhận một mã đối tượng như sau:

Bảng 2.2. Minh họa xác nhận trạng thái qua các lượt xử lý

Lượt	Quan sát	Bộ đếm	Kết quả
1	Không đội mũ	1	Chưa xác nhận
2	Mất phát hiện	0	Đặt lại chuỗi
3	Không đội mũ	1	Chưa xác nhận
4	Không đội mũ	2	Chưa xác nhận
5	Không đội mũ	3	Xác nhận, tạo bản ghi
6	Không đội mũ	4	Không tạo thêm bản ghi
7	Đội mũ	0	Giữ bản ghi đã xác nhận

Điều kiện này chặn một phát hiện sai đơn lẻ, nhưng báo nhầm kéo dài vẫn có thể được xác nhận. Nó bổ sung cho chất lượng nhận diện và liên kết, không thay thế các bước đó.

2.2.4.3. Giữ trạng thái đã xác nhận và ghi nhận một lần

Gọi $E_i(k)$ là trạng thái “đã từng được xác nhận không đội mũ”:

$$E_i^{(k)} = E_i^{(k-1)} \vee 1[r_i^{(k)} \geq K]. \quad (2.9)$$

Dấu nối trong công thức là phép “hoặc” logic; hàm chỉ báo $1[\cdot]$ bằng 1 khi điều kiện đúng. Khi E_i lần đầu chuyển sang 1, hệ thống tạo bản ghi, lưu minh chứng và đưa mã vào tập đã ghi nhận. Minh chứng lấy tại khung xác nhận, có thể muộn hơn lần đầu thấy trạng thái không mũ. Quan sát đội mũ về sau không xóa bản ghi đã xác nhận.

2.2.4.4. Đơn vị thống kê và ảnh hưởng của tham số

Tổng đối tượng trong video được tính theo mã trong lịch sử tracking; số trường hợp không đội mũ được tính theo các mã đã xác nhận, không cộng hợp giới hạn của từng khung. Tách một xe thành nhiều mã có thể gây đếm lặp, còn ghép nhiều xe thành một mã có thể làm giảm số đếm; cần kiểm tra với dữ liệu đối chiếu.

Bước lấy mẫu s và ngưỡng xác nhận K tác động đồng thời. Khi các lần phát hiện liên tục, thời gian từ quan sát đầu đến lần xác nhận thứ K là $(K-1)s/f$. Tăng K hoặc s có thể làm xác nhận muộn và bỏ sót đối tượng chỉ xuất hiện ngắn. Với ảnh tĩnh, không có chuỗi thời gian nên hệ thống xác định kết quả từ phát hiện và liên kết tại thời điểm phân tích, không áp dụng điều kiện nhiều khung hình.

2.3. Công nghệ và công cụ xây dựng hệ thống

Đề tài kết hợp công cụ huấn luyện mô hình, xử lý ảnh và video với công nghệ xây dựng ứng dụng web, nhằm tiếp nhận dữ liệu, lưu kết quả nhận diện và thống kê theo địa điểm, thời gian.

2.3.1. Python, Ultralytics và OpenCV

Python được dùng cho notebook huấn luyện và các chức năng xử lý phía máy chủ: nạp mô hình, nhận diện, liên kết vùng đầu với xe, tracking và tổng hợp kết quả. Ultralytics hỗ trợ nạp checkpoint, huấn luyện, đánh giá và chạy dự đoán bằng YOLO. Đầu ra gồm hộp giới hạn, nhãn lớp và độ tin cậy, cung cấp dữ liệu cho bước kiểm tra quan hệ giữa vùng đầu và xe máy.

OpenCV đảm nhiệm việc đọc ảnh, video, lấy khung hình, thay đổi kích thước, vẽ kết quả, cắt ảnh minh chứng và ghi video đầu ra. Trong quy trình chung, OpenCV chuẩn bị khung hình; Ultralytics thực hiện phát hiện; các chức năng Python liên kết, theo dõi và tổng hợp; sau đó OpenCV lưu kết quả trực

quan. Việc cùng sử dụng Python giúp tích hợp mô hình với ứng dụng web và cơ chế lưu trữ.

2.3.2. Google Colab và Google Drive trong huấn luyện mô hình

Google Colab là môi trường notebook trực tuyến dùng để chạy mã Python, có thể cung cấp GPU tùy phiên làm việc. Notebook của đề tài thực hiện các bước kết nối dữ liệu, kiểm tra môi trường, nạp checkpoint hai lớp, huấn luyện trên dữ liệu ba lớp và đánh giá kết quả. Google Drive lưu dữ liệu, trọng số ban đầu, checkpoint và các tệp đánh giá để hạn chế mất kết quả khi phiên Colab bị ngắt.

Khi báo cáo thực nghiệm, cần ghi thông tin từ lần chạy thực tế: phần cứng được cấp, phiên bản thư viện, số epoch hoàn thành và tập dữ liệu đánh giá. Checkpoint ba lớp được lựa chọn sau huấn luyện và đánh giá sẽ được đưa vào ứng dụng để phân tích ảnh, video tải lên.

2.3.3. Công nghệ xây dựng hệ thống

Ứng dụng web gồm backend, frontend và cơ sở dữ liệu. Backend được xây dựng bằng Python và FastAPI để cung cấp API quản lý tài khoản, dự án, địa điểm, tệp và phiên phân tích; kiểm tra dữ liệu đầu vào; chạy mô hình; lưu minh chứng và trả kết quả. Frontend giao tiếp với backend qua API HTTP theo hướng REST: dữ liệu quản lý, kết quả sử dụng JSON, còn ảnh và video tải lên bằng `multipart/form-data`. Với video xử lý lâu, backend chạy tác vụ nền; giao diện định kỳ truy vấn trạng thái để hiển thị tiến độ.

Frontend sử dụng React, TypeScript và Vite. React tổ chức giao diện thành các thành phần như biểu mẫu, danh sách tệp, biểu đồ và khu vực xem minh chứng; TypeScript mô tả cấu trúc dữ liệu và hỗ trợ phát hiện lỗi kiểu; Vite phục vụ phát triển và đóng gói giao diện. Các công nghệ này được dùng cho trang đăng nhập, quản lý dự án, địa điểm, tải dữ liệu, xem kết quả và thống kê.

Hệ thống dùng SQLite trong cấu hình mặc định và SQLAlchemy để thao tác dữ liệu. SQLite lưu dữ liệu trên tệp, phù hợp quy mô đồ án; SQLAlchemy quản lý các thực thể như người dùng, dự án, địa điểm, phiên phân tích, tệp và trường hợp được ghi nhận. Nội dung ảnh, video, minh chứng nằm trong thư mục trên máy chủ; cơ sở dữ liệu lưu thông tin mô tả và đường dẫn. Cách tổ chức này cho phép truy xuất lịch sử, kiểm tra kết quả và tổng hợp số liệu.

2.4. Các tiêu chí và chỉ số đánh giá

Đánh giá được thực hiện ở hai cấp. Cấp mô hình xem xét khả năng định vị và phân loại *helmet*, *no-helmet*, *bike*. Cấp hệ thống kiểm tra việc ghi nhận đúng

trường hợp cần tìm, hạn chế nhầm người đi bộ, đếm trùng và sai số thống kê. Tập validation phục vụ lựa chọn mô hình, tham số; tập test độc lập dùng để đánh giá cuối cùng. Nếu chỉ có validation, báo cáo phải ghi rõ giới hạn này.

2.4.1. Chỉ số IoU và cách xác định phát hiện đúng, báo nhầm, bỏ sót

2.4.1.1. Chỉ số IoU

IoU (*Intersection over Union*) đo mức chồng lấp giữa hộp dự đoán B_p và hộp đối chứng B_g :

$$\text{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|} \quad (2.10)$$

Giá trị nằm trong khoảng 0–1; càng cao, hai hộp càng phù hợp về vị trí và kích thước. IoU chỉ phản ánh hình học, nên một phát hiện đúng còn phải có nhãn lớp phù hợp.

2.4.1.2. Xác định TP, FP và FN

Tại ngưỡng độ tin cậy và IoU đã chọn:

Ký hiệu	Ý nghĩa	Cách xác định
TP (<i>True Positive</i>)	Phát hiện đúng	Dự đoán đúng lớp và ghép được với đối tượng đối chứng tại ngưỡng IoU
FP (<i>False Positive</i>)	Phát hiện sai hoặc thừa	Dự đoán không ghép được với đối tượng đối chứng phù hợp
FN (<i>False Negative</i>)	Bỏ sót	Đối tượng đối chứng không có dự đoán phù hợp được ghép

Mỗi đối tượng đối chứng chỉ được ghép với tối đa một dự đoán TP; các dự đoán dư có thể là FP. Dự đoán sai lớp có thể đồng thời tạo FP ở lớp dự đoán và FN ở lớp thực tế. Cần phân biệt cấp mô hình với cấp hệ thống: vùng đầu người đi bộ được phát hiện đúng là *no-helmet* vẫn trở thành báo nhầm ở cấp hệ thống nếu bị ghi nhận là trường hợp đi xe máy không đội mũ.

2.4.2. Precision, Recall và F1-score

2.4.2.1. Precision

Precision là tỷ lệ TP trong các dự đoán được đưa ra:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2.11)$$

Chỉ số cao cho thấy ít phát hiện sai. Ở cấp hệ thống, Precision của các trường hợp không đội mũ còn chịu tác động của bước liên kết và xác nhận qua video.

2.4.2.2. Recall

Recall là tỷ lệ TP trong các đối tượng thực tế:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2.12)$$

Recall cao cho thấy ít bỏ sót. Cần xem Recall của cả *no-helmet* và *bike*: nếu bỏ sót xe máy, hệ thống có thể không xác nhận được vùng đầu không mũ dù đã phát hiện đúng trạng thái.

2.4.2.3. F1-score

F1-score là trung bình điều hòa của Precision và Recall:

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} \quad (2.13)$$

F1 nằm trong khoảng 0–1. Khi mẫu số bằng 0, cần nêu quy ước của công cụ đánh giá. Ví dụ, với 18 TP, 2 FP và 3 FN, Precision bằng 0,90; Recall xấp xỉ 0,857 và F1 xấp xỉ 0,878. Đây chỉ là ví dụ cách tính.

Với mô hình nhiều lớp, báo cáo chỉ số của từng lớp và trung bình. Trung bình các F1 theo lớp không nhất thiết bằng F1 tính từ Precision trung bình và Recall trung bình; notebook của đề tài ưu tiên trung bình các F1 theo lớp do thư viện trả về. Ngoài ra, các chỉ số phụ thuộc ngưỡng độ tin cậy. Không nên mặc định Precision, Recall và F1 từ `val()` tương ứng với ngưỡng dự đoán 0,25, vì Ultralytics chọn điểm báo cáo theo đường cong đánh giá.

2.4.3. mAP@50 và mAP@50–95

2.4.3.1. AP và mAP

Khi thay đổi ngưỡng độ tin cậy, Precision và Recall tạo thành đường cong Precision–Recall. AP (*Average Precision*) tổng hợp chất lượng phát hiện của một lớp theo diện tích dưới đường cong với quy tắc nội suy của bộ đánh giá. Với C lớp và ngưỡng IoU τ , mAP là trung bình AP:

$$\text{mAP}_\tau = \frac{1}{C} \sum_{c=1}^C \text{AP}_{c,\tau} \quad (2.14)$$

2.4.3.2. mAP@50

mAP@50 dùng ngưỡng IoU 0,50. Chỉ số tổng hợp chất lượng phát hiện các lớp qua nhiều ngưỡng độ tin cậy, không phải tỷ lệ trường hợp không đội mũ được hệ thống xác định đúng. Chẳng hạn, mAP@50 bằng 0,90 không có nghĩa hệ thống xác định đúng 90% trường hợp cần ghi nhận.

2.4.3.3. mAP@50–95

mAP@50–95 là trung bình mAP tại 10 ngưỡng IoU từ 0,50 đến 0,95, bước 0,05:

$$\text{mAP@50–95} = \frac{1}{10} \sum_{k=0}^9 \text{mAP}_{0,50+0,05k} \quad (2.15)$$

Ngưỡng IoU cao yêu cầu vị trí hộp dự đoán chính xác hơn. Khi so sánh mô hình hai lớp với ba lớp, cần đánh giá hai lớp chung *helmet*, *no-helmet* trên cùng tập ảnh, cùng quy tắc nhãn và ánh xạ đúng ID. Lớp *bike* được báo cáo riêng; có thể trình bày thêm mAP tổng hợp ba lớp. Không thể chỉ so sánh trực tiếp một giá trị mAP trung bình hai lớp với một giá trị mAP trung bình ba lớp rồi kết luận mức cải tiến.

2.4.4. Ma trận nhầm lẫn, đường cong đánh giá và kết quả theo từng lớp

2.4.4.1. Ma trận nhầm lẫn

Ma trận nhầm lẫn cho thấy quan hệ giữa lớp đối chứng và lớp dự đoán. Trong biểu đồ Ultralytics, trục ngang là lớp thực tế, trục dọc là lớp dự đoán. Các ô đường chéo thể hiện ghép đúng; ô ngoài đường chéo thể hiện nhầm lớp. Cột *background* biểu diễn dự đoán không ghép được, còn hàng *background* biểu diễn đối tượng bị bỏ sót. Nên xem cả số lượng và tỷ lệ để nhận biết nhầm lẫn giữa *helmet*, *no-helmet* cũng như mức bỏ sót *bike*.

2.4.4.2. Các đường cong đánh giá

Đường cong	Nội dung thể hiện
Precision–Recall	Quan hệ giữa Precision và Recall
Precision–Confidence	Precision theo ngưỡng độ tin cậy
Recall–Confidence	Recall theo ngưỡng độ tin cậy
F1–Confidence	F1 theo ngưỡng độ tin cậy

Các đường cong hỗ trợ chọn ngưỡng cân bằng báo nhầm và bỏ sót. Ngưỡng nên được chọn trên validation, sau đó giữ cố định khi kiểm thử. Do mục tiêu của

đề tài là giảm nhảm người đi bộ, cần xem thêm kết quả cuối cùng sau bước liên kết, không chỉ đường cong của mô hình.

2.4.4.3. Kết quả theo từng lớp

Bảng đánh giá cần nêu số đối tượng đối chứng, Precision, Recall, F1, AP@50 và AP@50–95 của từng lớp, bên cạnh chỉ số trung bình. Cần đối chiếu trực quan một số ảnh dự đoán với nhãn thực tế, nhất là cảnh người đi bộ gần xe, nhiều phương tiện, đối tượng nhỏ và bị che khuất. Việc này giúp phát hiện hạn chế mà chỉ số trung bình có thể che lấp.

2.4.5. Đánh giá hệ thống: sai số đếm, thời gian xử lý và kiểm thử chức năng

2.4.5.1. Đánh giá ghi nhận đúng và sai số đếm

Kết quả hệ thống được đối chiếu với kiểm tra thủ công trên cùng ảnh hoặc video và cùng đơn vị đếm. Số lượt xe có người không đội mũ không đồng nhất với số người không đội mũ; tổng hợp phát hiện qua các khung hình cũng không phải số đối tượng duy nhất.

Với mẫu thứ i , gọi $\hat{n}_i$ là số lượng hệ thống ghi nhận và n_i là số lượng đối chứng. Sai số tuyệt đối và sai số tuyệt đối trung bình trên M mẫu là:

$$E_i = |\hat{N}_i - N_i| \quad (2.16)$$

$$MAE = \frac{1}{M} \sum_{i=1}^M |\hat{N}_i - N_i| \quad (2.17)$$

MAE nhỏ cho thấy số đếm gần đối chứng, nhưng tổng đúng chưa chứng minh từng trường hợp đều đúng: một báo nhảm và một bỏ sót có thể triệt tiêu nhau. Do đó, cần thống kê riêng ghi nhận đúng, báo nhảm, bỏ sót và đếm trùng. Bộ kiểm thử phải có cảnh chỉ có người đi bộ, người đi bộ gần xe, xe bị che khuất hoặc không được phát hiện.

2.4.5.2. Thời gian và tốc độ xử lý

Thời gian xử lý toàn trình được đo từ lúc máy chủ bắt đầu xử lý tệp đã tải lên đến khi lưu xong kết quả; bao gồm đọc dữ liệu, nhận diện, liên kết, tracking, ghi ảnh hoặc video và lưu thông tin. Nếu báo cáo tốc độ suy luận riêng của mô hình, sử dụng:

$$FPS_{\text{suy luận}} = \frac{N_{\text{khung suy luận}}}{T_{\text{suy luận}}} \quad (2.18)$$

Trong đó, $N_{\text{suy luận}}$ là số khung thực sự đưa qua mô hình, $T_{\text{suy luận}}$ là tổng thời gian suy luận tính bằng giây. FPS suy luận không thay thế thời gian xử

lý toàn hệ thống. Khi so sánh cần ghi phần cứng, độ phân giải, thời lượng video và bước lấy mẫu.

2.4.5.3. Kiểm thử chức năng

Kiểm thử chức năng dựa trên các thao tác sử dụng chính:

Bảng 2.3. Chức năng hệ thống

Nhóm chức năng	Nội dung cần kiểm tra
Quản lý dữ liệu	Tạo, cập nhật và truy xuất đúng dự án, địa điểm
Tải ảnh, video	Tiếp nhận tệp hợp lệ, thông báo dữ liệu không phù hợp
Xử lý dữ liệu	Cập nhật trạng thái, tiến độ, kết quả hoặc lỗi
Hiển thị kết quả	Hiển thị đúng ảnh, video, số lượng và minh chứng
Thống kê	Tổng hợp đúng theo địa điểm, thời gian đã chọn
Xuất kết quả	Tải được tệp kết quả phù hợp với phiên phân tích

Mỗi trường hợp kiểm thử cần ghi dữ liệu đầu vào, kết quả mong đợi, kết quả thực tế và trạng thái đạt hoặc chưa đạt. Các số liệu đo và so sánh hai phương án được trình bày trong chương thực nghiệm.

2.5. Kết luận chương 2

Chương 2 đã trình bày cơ sở phát hiện đối tượng bằng YOLO, dữ liệu gán nhãn, học chuyển giao và huấn luyện tinh chỉnh mô hình từ hai lớp lên ba lớp. Sau nhận diện, liên kết vùng đầu với xe máy, Centroid Tracking và xác nhận qua nhiều khung hình hỗ trợ giảm nhầm người đi bộ và hạn chế đếm trùng. Các công nghệ Python, Ultralytics, OpenCV, Google Colab, FastAPI, React, TypeScript, Vite, SQLAlchemy và SQLite được sử dụng theo vai trò tương ứng trong hệ thống.

Bộ tiêu chí gồm IoU, Precision, Recall, F1, mAP, ma trận nhầm lẫn, sai số đếm, tốc độ xử lý và kiểm thử chức năng cho phép đánh giá cả mô hình lẫn kết quả hệ thống. Đây là cơ sở để trình bày kiến trúc, cơ sở dữ liệu, quy trình hoạt động và thực nghiệm ở các chương tiếp theo.